APIM Security Demo — Presenter's Guide

> ***Purpose**: Click-by-click walkthrough for presenting this demo to customers. Each section tells you exactly what to show, why it matters, the customer value, how Microsoft does it better than alternatives, and which OWASP vulnerability is being addressed.*

Pre-Demo Checklist

Before starting, have these browser tabs ready:

Tab	URL	Purpose
1	Azure Portal — Resource Group	Show deployed resources
2	APIM Gateway — Test Console	Live API calls
3	GitHub Repository	Show code and CI/CD
4	GitHub PR #1	Show blocked insecure PR
5	Terminal / PowerShell	Live curl commands

Subscription Key (for live calls): `4ebbfcbaef1c4e07a512339b62d108ca` **Gateway URL**: `https://apim-security-demo-dev-exxlcmfwvdwzi.azure-api.net`

PHASE 1: The Problem — Why API Security Matters (5 min)

Click 1: Open the Backend API directly

What to show: Navigate to `https://products-api-backend.ambitiousrock-3b12f8ac.eastus.azurecontainerapps.io/api/products`

What happens: The API returns all product data — no authentication, no rate limiting, no security headers. Anyone on the internet can access this.

Why this matters: This is how most APIs start — a developer builds a REST API, deploys it, and ships it. No security. The OWASP API Security Top 10 reports that **API1: Broken Object Level Authorization** and **API2: Broken Authentication** are the #1 and #2 most exploited API vulnerabilities.

Customer value: "This is what your APIs look like without a gateway in front of them. Every endpoint is exposed, every response leaks server information, and there's nothing stopping an attacker from scraping your entire database."

Competitive edge: Unlike AWS API Gateway (which requires Lambda authorizers for JWT validation) or Kong (which needs plugin installation), Azure APIM provides **built-in JWT validation, rate limiting, and security headers as declarative XML policies** — zero code required.

Click 2: Try creating a fake product

What to show: Run in terminal:

```
curl -X POST https://products-api-backend.ambitiousrock-
3b12f8ac.eastus.azurecontainerapps.io/api/products \
  -H "Content-Type: application/json" \
  -d '{"name":"HACKED","category":"Injected","price":0}'
```

What happens: Product is created successfully. No validation, no auth, no audit trail.

OWASP vulnerability: **API3 — Broken Object Property Level Authorization**. The attacker can set any field (including price to 0) because there's no schema validation or authorization check on which properties can be written.

Customer value: "An attacker just created a product with price $0 in your database. In a real scenario, this could be financial fraud, data poisoning, or a stepping stone for privilege escalation."

PHASE 2: The Solution — Azure APIM as a Security Layer (10 min)

Click 3: Show the APIM resource in Azure Portal

What to show: Navigate to Azure Portal > Resource Group `rg-apim-security-demo` > Click on APIM instance

What to highlight:

- **TLS 1.2 minimum** enforced (Settings > Protocols + Ciphers)
- **Managed Identity** enabled (for secure backend authentication)
- **Application Insights** connected (full request/response logging)
- **Developer Portal** available for API consumers

Why this matters: APIM is not just a reverse proxy — it's a full security control plane. Every request flows through the policy pipeline: Inbound → Backend → Outbound → On-Error.

Competitive edge:

Feature	Azure APIM	AWS API Gateway	Kong	Apigee
Built-in JWT validation	Yes (declarative XML)	No (requires Lambda)	Plugin required	Yes
Policy inheritance (Global → API → Operation)	Yes, 3 levels	No	Limited	Yes, but complex
SARIF security scanning	Yes (our custom scanner)	No built-in	No	No
GitHub Copilot integration	Native (Agentic Workflows)	No	No	No
Infrastructure as Code	Bicep + ARM native	CloudFormation	Helm charts	Terraform
Developer Portal	Built-in, customizable	No	Kong Developer Portal	Integrated Portal

---

Click 4: Demonstrate Subscription Key Enforcement

What to show: Run in terminal:

```
# Without subscription key — BLOCKED
curl -s https://apim-security-demo-dev-exxlcmfwvdwzi.azure-api.net/products/api/products
```

What happens: HTTP 401 — "Access denied due to missing subscription key"

OWASP vulnerability: **API2 — Broken Authentication**. Without a subscription key, the caller cannot prove their identity. APIM enforces this before any policy even runs.

Customer value: "The very first thing APIM does is check whether the caller has a valid subscription. This is your first line of defense — every API consumer must register and get a key. You can revoke access instantly by disabling their subscription."

How the policy protects: In `api-definition.bicep`, the API is defined with `subscriptionRequired: true`. This is enforced at the platform level — no policy XML needed. The subscription key identifies the caller, enabling per-caller rate limiting and usage analytics.

Click 5: Demonstrate JWT Token Enforcement

What to show: Run in terminal:

```
# With subscription key but no JWT — BLOCKED
curl -s -H "Ocp-Apim-Subscription-Key: 4ebbfcbaef1c4e07a512339b62d108ca" \
  https://apim-security-demo-dev-exxlcmfwvdwzi.azure-api.net/products/api/products
```

What happens: HTTP 401 — "Access denied. Valid JWT token required."

OWASP vulnerability: **API2 — Broken Authentication**. Even with a valid subscription key, the caller must also present a valid OAuth 2.0 / JWT token issued by Azure Active Directory. This is two-factor API authentication.

Customer value: "Subscription keys identify the application. JWT tokens identify the user. Together, you know exactly WHO is calling your API and WHICH application they're using. This is zero-trust API security."

How the policy protects: The `validate-jwt` policy in `api-level-policy.xml` checks:

- Token is present in the `Authorization` header
- Token is signed by your Azure AD tenant (via OpenID Connect discovery)
- Token has not expired (`require-expiration-time="true"`)
- Token audience matches your API's App Registration
- Token issuer matches your Azure AD tenant

```
<validate-jwt header-name="Authorization"
              failed-validation-httpcode="401"
              require-expiration-time="true"
              require-signed-tokens="true">
    <openid-config url="https://login.microsoftonline.com/{{tenant-id}}/v2.0/.well-
known/openid-configuration" />
    <audiences>
        <audience>{{api-audience}}</audience>
```

```
    </audiences>
</validate-jwt>
```

Click 6: Show Rate Limiting Headers

What to show: Run in terminal:

```
# Check response headers
curl -s -D- -H "Ocp-Apim-Subscription-Key: 4ebbfcbaef1c4e07a512339b62d108ca" \
  https://apim-security-demo-dev-exxlcmfwvdwzi.azure-api.net/products/api/products 2>&1 |
head -15
```

What to highlight: `X-RateLimit-Remaining: 59` and `X-RateLimit-Limit: 60` headers.

OWASP vulnerability: **API4 — Unrestricted Resource Consumption**. Without rate limiting, an attacker can:

- DDoS your API with millions of requests
- Exhaust your backend database connections
- Run up your cloud bill (API4 is also a financial attack)
- Scrape your entire dataset via pagination abuse

Customer value: "APIM enforces 60 requests per minute per IP at the global level, and 100 per minute per subscription at the API level. When the limit is hit, the caller gets HTTP 429 (Too Many Requests) with a `Retry-After` header. Your backend never sees the excess traffic."

How the policy protects: Two layers of rate limiting in `global-policy.xml` and `api-level-policy.xml`:

```
<!-- Global: per IP address -->
<rate-limit-by-key calls="60" renewal-period="60"
                   counter-key="@(context.Request.IpAddress)" />

<!-- API-level: per subscription -->
<rate-limit-by-key calls="100" renewal-period="60"
                   counter-key="@(context.Subscription.Id)" />
```

Competitive edge: AWS API Gateway rate limiting is per-stage only (no per-IP, no per-subscription). Kong requires the rate-limiting plugin. APIM's `rate-limit-by-key` supports arbitrary counter keys using C# expressions — rate limit by IP, subscription, JWT claim, header value, or any combination.

Click 7: Show Security Headers

What to show: Point out the response headers from the previous curl:

- `X-Content-Type-Options: nosniff`
- `X-Frame-Options: DENY`
- `Strict-Transport-Security: max-age=31536000`
- `Referrer-Policy: strict-origin-when-cross-origin`
- Note the ABSENCE of `X-Powered-By` and `Server` headers (stripped!)

OWASP vulnerability: **API8 — Security Misconfiguration**. Default server headers reveal:

- What web framework is used (Express, ASP.NET, etc.)

- What server software is running (nginx, IIS, etc.)
- This gives attackers a roadmap of known CVEs to exploit

Customer value: "APIM strips all server fingerprinting headers and adds security headers that protect against clickjacking (X-Frame-Options), MIME sniffing (X-Content-Type-Options), and protocol downgrade attacks (HSTS). This happens automatically on every response — your developers don't have to remember to add these."

How the policy protects: The global outbound policy:

```
<!-- REMOVE fingerprinting -->
<set-header name="X-Powered-By" exists-action="delete" />
<set-header name="Server" exists-action="delete" />

<!-- ADD security headers -->
<set-header name="X-Content-Type-Options" exists-action="override">
    <value>nosniff</value>
</set-header>
<set-header name="Strict-Transport-Security" exists-action="override">
    <value>max-age=31536000; includeSubDomains; preload</value>
</set-header>
```

PHASE 3: Automated Security — GitHub Agentic Workflow (10 min)

Click 8: Show the GitHub Repository

What to show: Navigate to `https://github.com/sautalwar/how_APIM_works`

What to highlight:

- `policies/` folder — all security policies as code (version controlled!)
- `security-scanner/` — custom Python scanner with 18 OWASP rules
- `.github/workflows/` — CI/CD automation
- `.github/copilot/agentic-security-review.md` — AI security reviewer

Customer value: "Everything is code. Policies are not configured through a portal and forgotten — they live in Git, they're reviewed in PRs, they're scanned automatically, and they're deployed through CI/CD. This is GitOps for API security."

Competitive edge: No other API gateway vendor provides this level of GitHub-native security automation. AWS API Gateway policies are configured via Console/CloudFormation with no built-in scanning. Kong plugins are configured via Admin API. APIM + GitHub is the only solution where **AI reviews your security policies before they're deployed**.

Click 9: Open Pull Request #1

What to show: Navigate to `https://github.com/sautalwar/how_APIM_works/pull/1`

What to highlight:

1. The PR title says "Add public API policy for partner integration" — sounds innocent
2. The policy file removes JWT authentication and uses wildcard CORS
3. **The CI check FAILED** — blocked by the security scanner

Why this matters: A well-meaning developer might relax security "temporarily" for partner onboarding. Without automated scanning, this goes to production. With our scanner, it's caught immediately.

OWASP vulnerabilities caught:

Finding	Severity	OWASP	What the scanner detected
AUTH001	CRITICAL	API2	No `validate-jwt` — API is completely unauthenticated
CORS001	CRITICAL	API8	Wildcard origin `*` — any website can call this API
CORS002	CRITICAL	API8	`allow-credentials="true"` with wildcard — worst possible CORS configuration
RATE001	HIGH	API4	No rate limiting — vulnerable to DDoS and scraping
ERR001	HIGH	API8	No on-error section — stack traces may leak to attackers
DATA001	HIGH	API4	No request size limit — resource exhaustion attack vector
HDR001-4	MEDIUM	API8	Missing security headers — server fingerprinting exposed

Customer value: "This PR was blocked in under 2 minutes. The scanner found 3 critical and 4 high-severity OWASP violations. The developer gets immediate feedback on exactly what's wrong and how to fix it. No security review bottleneck — the automation handles it."

Click 10: Show the CI Workflow Details

What to show: Click on the failed check "Policy Security Scan" > View job details

What to highlight:

1. **Lint & Validate** — passed (XML is syntactically valid)
2. **Policy Security Scan** — FAILED (OWASP violations found)
3. **Bicep What-If** — SKIPPED (blocked by security failure)

Why this matters: The pipeline has a gate. Even though the XML is valid and the Bicep would deploy successfully, the security scan prevents deployment. This is "shift-left security" — catch issues at the PR level, not in production.

Customer value: "Your deployment pipeline has three gates: syntax validation, security scanning, and infrastructure preview. If any gate fails, nothing deploys. This means a security vulnerability can NEVER reach production through the normal development workflow."

Competitive edge: This is a unique Microsoft advantage — **GitHub Actions + APIM + Copilot** is a vertically integrated stack. Competitors must stitch together 3-4 different vendors:

- AWS: API Gateway + CodePipeline + custom Lambda scanner + no AI review
- Kong: Kong Gateway + Jenkins/CircleCI + custom plugin + no AI review
- Apigee: Apigee + Cloud Build + apigeelint + no AI review
- **Microsoft: APIM + GitHub Actions + Custom Scanner + Copilot Agentic Workflow** — all from one vendor

---

Click 11: Show the Security Tab (SARIF Results)

What to show: Navigate to the repository's Security tab > Code scanning alerts

What to highlight: The scanner's findings are uploaded as SARIF (Static Analysis Results Interchange Format), so they appear in GitHub's native security dashboard alongside CodeQL findings.

Customer value: "Your API security findings live alongside your code security findings. One dashboard. One workflow. Your security team doesn't need a separate tool — everything is in GitHub Advanced Security."

Click 12: Show the Agentic Workflow Definition

What to show: Open `.github/copilot/agentic-security-review.md` in the repository

What to highlight: This file defines an AI-powered security reviewer that:

- Automatically reviews PRs touching `policies/` or `infra/` files
- Checks for all 10 OWASP API Security Top 10 violations
- Provides specific remediation with XML code examples
- Approves PRs only when all security checks pass

Customer value: "This is the future of security review. Instead of waiting days for a human security expert to review API policies, an AI agent reviews them in minutes. It knows every OWASP vulnerability, it never gets tired, and it provides specific fix suggestions with code examples."

Competitive edge: This is ONLY possible with GitHub Copilot. No other platform has AI-powered code review that understands API security policies at this level. This is the Microsoft moat — Azure + GitHub + Copilot is an unbeatable combination that no competitor can replicate.

PHASE 4: The Fix — Closing the Loop (5 min)

Click 13: Explain the Fix Path

What to tell the customer: "Here's what happens next in a real workflow:"

1. Developer sees the failed check on their PR
2. They read the scanner's findings — exact rule ID, OWASP reference, fix suggestion
3. They update the policy: add JWT validation, restrict CORS, add rate limiting
4. They push the fix — CI re-runs automatically
5. Scanner passes — all green
6. Copilot Agentic Workflow approves the PR
7. PR is merged to main
8. CD pipeline deploys the updated policy to APIM
9. The API is now secure — zero downtime, full audit trail in Git

Customer value: "The entire loop — from insecure PR to secure deployment — is automated. No manual security reviews. No deployment delays. No 'it works on my machine.' Every policy change is version-controlled, scanned, AI-reviewed, and deployed through CI/CD."

Key Talking Points for Customers

Why Microsoft?

1. **Vertically Integrated**: Azure APIM + GitHub + Copilot — one vendor, one platform, one support contract
2. **Enterprise-Grade**: APIM handles billions of API calls for Fortune 500 companies
3. **AI-Native**: Copilot Agentic Workflows bring AI to security review — not a bolt-on, but native
4. **Compliance-Ready**: APIM is SOC 2, ISO 27001, HIPAA, FedRAMP certified

5. **Zero-Trust Architecture**: Managed identity, JWT validation, subscription keys — defense in depth

Why Not Competitors?

Objection	Response
"We use AWS API Gateway"	AWS requires Lambda functions for JWT validation (extra cost, extra code, extra maintenance). APIM does it in 5 lines of XML. No Lambda cold-start delays.
"We use Kong"	Kong is powerful but requires plugin management, separate infrastructure, and has no native AI review. APIM is a managed service — Azure handles scaling, patching, and high availability.
"We use Apigee"	Apigee is Google's offering but lacks GitHub-native CI/CD integration and has no equivalent to Copilot Agentic Workflows. The migration path from Apigee to APIM is well-documented.
"We build our own gateway"	Custom gateways are a security liability — every CVE is your responsibility. APIM gets Microsoft's security team, threat intelligence, and 24/7 monitoring for free.

The ROI Conversation

- **Time saved**: Security review from 3 days → 2 minutes (automated scanning)
- **Cost avoided**: One API breach costs $3.6M average (IBM 2024). APIM Developer tier costs ~$49/month.
- **Developer velocity**: No security review bottleneck — developers merge PRs same-day
- **Compliance**: Full audit trail in Git — every policy change is tracked, reviewed, and approved
- **Incident response**: Correlate API issues instantly with Application Insights + Log Analytics

---

OWASP API Top 10 — Complete Policy Protection Map

#	OWASP Vulnerability	Real-World Impact	APIM Policy Protection	Policy File
API1	Broken Object Level Authorization	Attacker accesses other users' data by changing IDs	`validate-jwt` with `required-claims` for object ownership	`operation-level-policy.xml`
API2	Broken Authentication	Attacker calls API without valid credentials	`validate-jwt` + `subscriptionRequired: true`	`api-level-policy.xml`
API3	Broken Object Property Level Authorization	Attacker modifies sensitive fields (price, role)	`validate-content` with JSON schema validation	`fragments/request-validation.xml`
API4	Unrestricted Resource Consumption	DDoS, scraping, cloud bill explosion	`rate-limit-by-key` + `quota-by-key`	`global-policy.xml`, `api-level-policy.xml`

API5	Broken Function Level Authorization	Non-admin calls admin-only endpoints	`validate-jwt` with `required-claims` role check	`operation-level-policy.xml`
API6	Unrestricted Access to Sensitive Business Flows	Automated ticket scalping, credential stuffing	`rate-limit-by-key` with custom counters + CAPTCHA integration	`global-policy.xml`
API7	Server Side Request Forgery (SSRF)	Attacker forces backend to call internal services	`set-backend-service` with allowlisted URLs + IP filtering	`fragments/ip-filtering.xml`
API8	Security Misconfiguration	Exposed server headers, verbose errors, weak TLS	`set-header` (delete/override) + `on-error` safe responses	`global-policy.xml`
API9	Improper Inventory Management	Shadow APIs, deprecated endpoints still accessible	API versioning + `<api>` definitions in Bicep	`infra/modules/api-definition.bicep`
API10	Unsafe Consumption of 3rd Party APIs	Trusting external API responses without validation	`validate-content` + `send-request` with response validation	`fragments/request-validation.xml`

Post-Demo Follow-Up

Leave-behind materials (the PDF files generated with this guide):

1. APIM Architecture Overview (01-apim-architecture.pdf)
2. Policy Pipeline Deep Dive (02-policy-pipeline.pdf)
3. OWASP API Top 10 Mitigations (03-owasp-api-top10.pdf)
4. Security Automation with GitHub (04-security-automation.pdf)
5. Demo Walkthrough (05-demo-walkthrough.pdf)
6. Azure Resource Inventory (06-azure-resource-inventory.pdf)
7. **This Presenter's Guide** (07-presenters-guide.pdf)

Next steps to offer the customer:

1. "Let us do a free API security assessment of your current APIs"
2. "We can set up a proof-of-concept with your APIs in APIM in 1-2 weeks"
3. "Our team can help migrate your existing API gateway to APIM"
4. "GitHub Enterprise includes Copilot — the Agentic Workflow is ready to use today"